

Звіт

до Лабораторної роботи №13

Обробка тексту і регулярні вирази в Java

Студент: Колісніченко Артем Олександрович

Група: КН-924в

Викладач: Савченко В. М.

Рік: 2026

1. Мета роботи

Метою лабораторної роботи є отримання практичних навичок обробки текстових даних у Java.

У роботі потрібно використати базові операції класу String, регулярні вирази, класи Pattern і Matcher, а також StringBuilder для формування підсумкового текстового звіту.

2. Варіант роботи

Варіант 5 — інтернет-магазин

Для обраного варіанта потрібно обробити текстові записи про товари, кількість і ціну. Також потрібно перевірити артикул товару, вилучити числові поля та сформувати чек або підсумковий звіт.

3. Опис предметної області

У лабораторній роботі реалізовано просту систему обробки записів інтернет-магазину.

Вхідні дані зберігаються у текстовому файлі orders.txt. Кожен рядок описує один товарний запис: артикул, назву товару, кількість і ціну.

Програма читає файл, нормалізує рядки, перевіряє формат запису, вилучає потрібні поля через регулярний вираз, перетворює коректні рядки на об'єкти та формує підсумковий звіт.

4. Класи проекту

Клас	Призначення
ProductOrder	Модель одного товарного запису.
OrderReport	Модель підсумкового звіту після обробки рядків.
OrderTextProcessor	Сервіс для нормалізації, перевірки, розбору рядків і побудови звіту.
Main	Демонстраційний клас, який читає файл і виводить результат.

5. Клас ProductOrder

Клас ProductOrder описує один товарний запис інтернет-магазину.

Поля класу:

- article — артикул товару;
- productName — назва товару;
- quantity — кількість товару;
- price — ціна одного товару.

Також у класі реалізовано метод getTotalPrice(), який обчислює повну суму за товар.

```
public double getTotalPrice() {  
    return quantity * price;  
}
```

6. Клас OrderReport

Клас OrderReport зберігає підсумкову інформацію після обробки текстового файлу.

Поля класу:

- validRecords — кількість коректних записів;
- invalidRecords — кількість некоректних записів;
- totalAmount — загальна сума замовлення.

7. Клас OrderTextProcessor

Клас OrderTextProcessor виконує основну логіку лабораторної роботи.

Він відповідає за:

- нормалізацію тексту;
- перевірку формату запису;
- вилучення полів через регулярний вираз;
- перетворення рядка на об'єкт ProductOrder;
- підрахунок коректних і некоректних записів;
- обчислення загальної суми;
- побудову текстового звіту.

8. Вхідний файл

Вхідні дані зберігаються у файлі:

src/main/resources/orders.txt

Приклад вмісту файлу:

ART-1001; Ноутбук Lenovo; QTY=1; PRICE=42000.50

ART-1002; Миша Logitech; QTY=2; PRICE=850.00

ART-1003; Клавіатура HyperX; QTY=1; PRICE=3200.00

ART-1004; Монітор Samsung; QTY=1; PRICE=7800.99

BAD-12; Некоректний товар; QTY=1; PRICE=100.00

ART-1005; Навушники Logitech; QTY=2; PRICE=2500.00

ART-1006; USB кабель; QTY=3; PRICE=150.75

Файл містить як коректні, так і некоректні записи. Некоректні записи не перетворюються на об'єкти, але враховуються у підсумковому звіті.

9. Формат запису

Коректний запис має такий формат:

ART-0000; Назва товару; QTY=Кількість; PRICE=Ціна

Приклад коректного запису:

ART-1001; Ноутбук Lenovo; QTY=1; PRICE=42000.50

Приклад некоректного запису:

BAD-12; Некоректний товар; QTY=1; PRICE=100.00

Цей запис є некоректним, тому що артикул не відповідає формату ART-0000.

10. Регулярний вираз

Для перевірки та розбору рядка використовується регулярний вираз:

```
^(ART-\d{4});\s*(.+?);\s*QTY=(\d+);\s*PRICE=(\d+(?:\.\d{1,2})?)$
```

Цей вираз перевіряє структуру товарного запису та вилучає потрібні поля.

Група	Значення
1	Артикул товару.
2	Назва товару.
3	Кількість товару.
4	Ціна товару.

11. Використання String

Для очищення рядків використано базові операції класу String.

```
public static String normalizeText(String line) {
```

```
if (line == null) {  
    return "";  
}  
  
return line.trim().replaceAll("\\s+", " ");  
}
```

Метод `trim()` прибирає зайві пробіли на початку та в кінці рядка. Метод `replaceAll()` замінює декілька пробілів одним.

12. Використання Pattern і Matcher

Для роботи з регулярним виразом використано класи `Pattern` і `Matcher`.

```
Matcher matcher = ORDER_PATTERN.matcher(line);
```

```
if (!matcher.matches()) {  
    throw new IllegalArgumentException("Некоректний запис товару: " +  
rawLine);  
}
```

```
String article = matcher.group(1);  
String productName = matcher.group(2).trim();  
int quantity = Integer.parseInt(matcher.group(3));  
double price = Double.parseDouble(matcher.group(4));
```

Метод `matcher.group()` використовується для вилучення окремих частин рядка. Після цього кількість і ціна перетворюються на числові типи.

13. Використання StringBuilder

Для формування підсумкового звіту використано `StringBuilder`. Це дозволяє зручно створювати багаторядковий текст.

```
StringBuilder builder = new StringBuilder();
```

```

builder.append("Звіт інтернет-магазину\n");
builder.append("Коректних записів:
").append(report.getValidRecords()).append('\n');
builder.append("Некоректних записів:
").append(report.getInvalidRecords()).append('\n');
builder.append("Загальна сума: ").append(report.getTotalAmount()).append("
грн\n");

```

14. UML-діаграма

Для проєкту було створено UML-діаграму. Вона показує модельні класи, сервіс текстової обробки та демонстраційний клас.

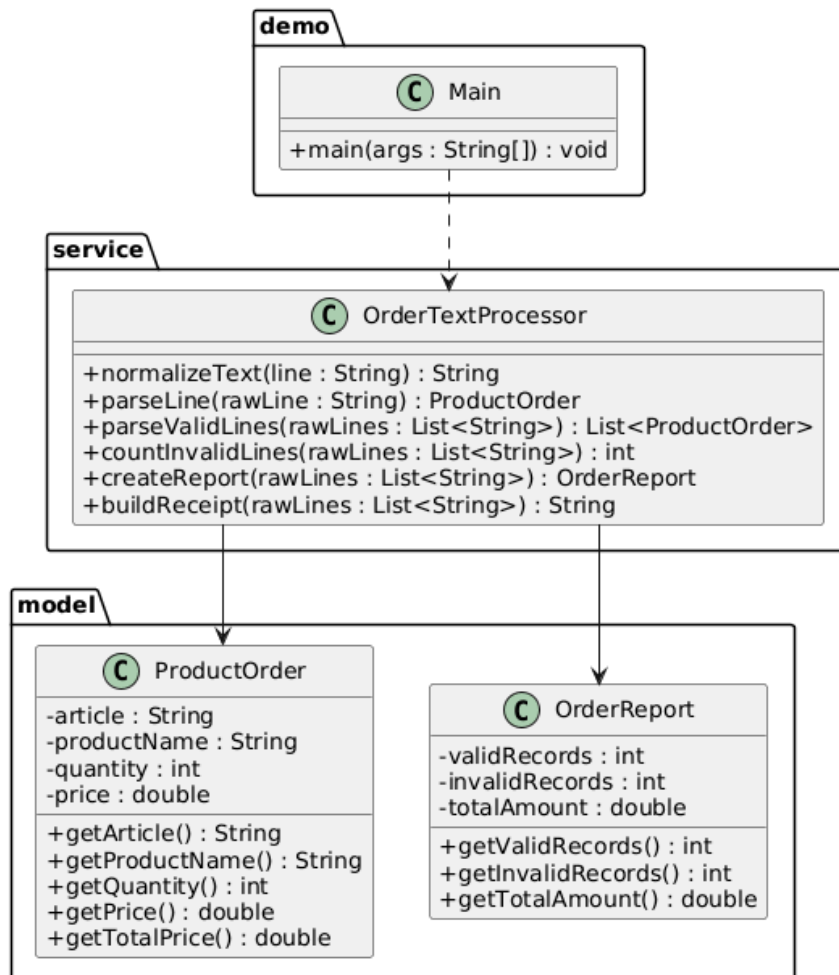


Рисунок 1 — UML-діаграма класів лабораторної роботи

15. Демонстрація роботи програми

У класі Main виконується читання файлу orders.txt з ресурсів проєкту.

Після цього створюється об'єкт OrderTextProcessor, який формує підсумковий звіт. Результат виводиться у консоль.

```
Звіт інтернет-магазину
Коректних записів: 6
Некоректних записів: 1
Загальна сума: 60153.74 грн

Товари:
- ART-1001 | Ноутбук Lenovo | 1 шт. | 42000.50 грн | сума: 42000.50 грн
- ART-1002 | Миша Logitech | 2 шт. | 850.00 грн | сума: 1700.00 грн
- ART-1003 | Клавіатура HyperX | 1 шт. | 3200.00 грн | сума: 3200.00 грн
- ART-1004 | Монітор Samsung | 1 шт. | 7800.99 грн | сума: 7800.99 грн
- ART-1005 | Навушники Logitech | 2 шт. | 2500.00 грн | сума: 5000.00 грн
- ART-1006 | USB кабель | 3 шт. | 150.75 грн | сума: 452.25 грн

Process finished with exit code 0
```

Рисунок 2 — результат запуску програми

16. Тестування

Для перевірки роботи програми були створені модульні тести JUnit Jupiter.

Тестові класи:

- OrderTextProcessorTest;
- MainTest.

Тести перевіряють:

- нормалізацію тексту;
- розбір коректного запису;
- відхилення некоректного артикулу;
- пропуск некоректних записів;
- підрахунок некоректних рядків;
- створення підсумкового звіту;

- формування текстового чека;
- запуск Main без помилок.

```

✓ OrderTextProcessorTest (ua.kl 48 ms)
✓ parseLineShouldCreateProc 26 ms
✓ normalizeTextShouldRemove 1 ms
✓ parseValidLinesShouldSkip 2 ms
✓ createReportShouldCalculate 2 ms
✓ buildReceiptShouldContain 14 ms
✓ parseLineShouldRejectInvalid 2 ms
✓ countInvalidLinesShouldReturn 1 ms

✓ 7 tests passed 7 tests total, 48 ms

C:\Users\bxqqe\jdk\ms-21.0.11\bin\java.exe ...

Process finished with exit code 0

```

Рисунок 3 — результат виконання тестів

17. Висновок

У ході виконання лабораторної роботи було реалізовано програму для обробки текстових записів інтернет-магазину.

Програма читає дані з текстового файлу, нормалізує рядки, перевіряє їхній формат за допомогою регулярного виразу, вилучає потрібні поля через Pattern і Matcher, перетворює записи на об'єкти предметної області та формує підсумковий звіт.

У роботі використано базові операції String, регулярні вирази, StringBuilder, файлове читання та модульне тестування.

Отже, лабораторна робота показує повний цикл текстової обробки: від сирого текстового файлу до структурованих об'єктів і готового звіту.